

```
In [1]: import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np
```

```
In [2]: # 1. DATA ACQUISITION
print("Loading data...")
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_test_images = X_test
```

Loading data...

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 2s 0us/step

```
In [3]: # 2. PREPROCESSING & NORMALIZATION
X_train = X_train.reshape(-1, 784) / 255.0
X_test = X_test.reshape(-1, 784) / 255.0
```

```
In [8]: # 3. LABEL ENCODING (One-Hot Encoding)
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

```
In [10]: # 4. MODEL ARCHITECTURE & TRAINING
with tf.device('/GPU:0'):
    model = Sequential([
        Dense(128, activation='relu', input_shape=(784,)),
        Dense(64, activation='relu'),
        Dense(10, activation='softmax')
    ])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
print("\nStarting training...")
```

Starting training...

C:\Users\aida-lab\AppData\Local\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:106: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
In [12]: history = model.fit(
X_train, y_train_cat,
epochs=50,
batch_size=32,
validation_split=0.1,
verbose=1
)
```

Epoch 1/50
1688/1688 ————— **10s** 5ms/step - accuracy: 0.9247 - loss: 0.2562 - val_accuracy: 0.9667 - val_loss: 0.1117

Epoch 2/50
1688/1688 ————— **9s** 4ms/step - accuracy: 0.9667 - loss: 0.1094 - val_accuracy: 0.9697 - val_loss: 0.1074

Epoch 3/50
1688/1688 ————— **11s** 4ms/step - accuracy: 0.9771 - loss: 0.0733 - val_accuracy: 0.9760 - val_loss: 0.0828

Epoch 4/50
1688/1688 ————— **8s** 5ms/step - accuracy: 0.9818 - loss: 0.0570 - val_accuracy: 0.9792 - val_loss: 0.0777

Epoch 5/50
1688/1688 ————— **6s** 4ms/step - accuracy: 0.9862 - loss: 0.0429 - val_accuracy: 0.9798 - val_loss: 0.0692

Epoch 6/50
1688/1688 ————— **13s** 5ms/step - accuracy: 0.9881 - loss: 0.0359 - val_accuracy: 0.9752 - val_loss: 0.0995

Epoch 7/50
1688/1688 ————— **6s** 3ms/step - accuracy: 0.9912 - loss: 0.0273 - val_accuracy: 0.9770 - val_loss: 0.0853

Epoch 8/50
1688/1688 ————— **12s** 4ms/step - accuracy: 0.9912 - loss: 0.0266 - val_accuracy: 0.9750 - val_loss: 0.1035

Epoch 9/50
1688/1688 ————— **11s** 5ms/step - accuracy: 0.9925 - loss: 0.0224 - val_accuracy: 0.9817 - val_loss: 0.0876

Epoch 10/50
1688/1688 ————— **8s** 3ms/step - accuracy: 0.9941 - loss: 0.0176 - val_accuracy: 0.9755 - val_loss: 0.1118

Epoch 11/50
1688/1688 ————— **12s** 5ms/step - accuracy: 0.9947 - loss: 0.0154 - val_accuracy: 0.9835 - val_loss: 0.0862

Epoch 12/50
1688/1688 ————— **10s** 4ms/step - accuracy: 0.9944 - loss: 0.0161 - val_accuracy: 0.9782 - val_loss: 0.1089

Epoch 13/50
1688/1688 ————— **9s** 4ms/step - accuracy: 0.9949 - loss: 0.0154 - val_accuracy: 0.9785 - val_loss: 0.1181

Epoch 14/50
1688/1688 ————— **8s** 5ms/step - accuracy: 0.9949 - loss: 0.0142 - val_accuracy: 0.9807 - val_loss: 0.1084

Epoch 15/50
1688/1688 ————— **9s** 4ms/step - accuracy: 0.9963 - loss: 0.0117 - val_accuracy: 0.9802 - val_loss: 0.1093

Epoch 16/50
1688/1688 ————— **13s** 5ms/step - accuracy: 0.9957 - loss: 0.0141 - val_accuracy: 0.9818 - val_loss: 0.0919

Epoch 17/50
1688/1688 ————— **9s** 4ms/step - accuracy: 0.9962 - loss: 0.0111 - val_accuracy: 0.9820 - val_loss: 0.0975

Epoch 18/50
1688/1688 ————— **8s** 5ms/step - accuracy: 0.9963 - loss: 0.0109 - val_accuracy: 0.9780 - val_loss: 0.1242

Epoch 19/50
1688/1688 ————— **7s** 4ms/step - accuracy: 0.9968 - loss: 0.0091 - val_accuracy: 0.9802 - val_loss: 0.1134

Epoch 20/50
1688/1688 ————— **8s** 5ms/step - accuracy: 0.9967 - loss: 0.0098 - val_accuracy: 0.9788 - val_loss: 0.1175

Epoch 21/50
1688/1688 ————— 6s 3ms/step - accuracy: 0.9973 - loss: 0.0086 - val_accuracy: 0.9785 - val_loss: 0.1262

Epoch 22/50
1688/1688 ————— 12s 4ms/step - accuracy: 0.9964 - loss: 0.0106 - val_accuracy: 0.9827 - val_loss: 0.1039

Epoch 23/50
1688/1688 ————— 12s 5ms/step - accuracy: 0.9976 - loss: 0.0077 - val_accuracy: 0.9808 - val_loss: 0.1155

Epoch 24/50
1688/1688 ————— 8s 3ms/step - accuracy: 0.9968 - loss: 0.0096 - val_accuracy: 0.9817 - val_loss: 0.1146

Epoch 25/50
1688/1688 ————— 12s 4ms/step - accuracy: 0.9969 - loss: 0.0100 - val_accuracy: 0.9822 - val_loss: 0.1213

Epoch 26/50
1688/1688 ————— 6s 4ms/step - accuracy: 0.9977 - loss: 0.0072 - val_accuracy: 0.9797 - val_loss: 0.1358

Epoch 27/50
1688/1688 ————— 11s 4ms/step - accuracy: 0.9973 - loss: 0.0091 - val_accuracy: 0.9803 - val_loss: 0.1406

Epoch 28/50
1688/1688 ————— 6s 3ms/step - accuracy: 0.9982 - loss: 0.0058 - val_accuracy: 0.9778 - val_loss: 0.1617

Epoch 29/50
1688/1688 ————— 11s 4ms/step - accuracy: 0.9976 - loss: 0.0079 - val_accuracy: 0.9802 - val_loss: 0.1335

Epoch 30/50
1688/1688 ————— 9s 3ms/step - accuracy: 0.9976 - loss: 0.0078 - val_accuracy: 0.9827 - val_loss: 0.1355

Epoch 31/50
1688/1688 ————— 6s 3ms/step - accuracy: 0.9977 - loss: 0.0078 - val_accuracy: 0.9818 - val_loss: 0.1399

Epoch 32/50
1688/1688 ————— 13s 5ms/step - accuracy: 0.9981 - loss: 0.0057 - val_accuracy: 0.9803 - val_loss: 0.1434

Epoch 33/50
1688/1688 ————— 6s 3ms/step - accuracy: 0.9972 - loss: 0.0091 - val_accuracy: 0.9802 - val_loss: 0.1537

Epoch 34/50
1688/1688 ————— 12s 5ms/step - accuracy: 0.9981 - loss: 0.0067 - val_accuracy: 0.9823 - val_loss: 0.1525

Epoch 35/50
1688/1688 ————— 10s 4ms/step - accuracy: 0.9978 - loss: 0.0072 - val_accuracy: 0.9802 - val_loss: 0.1670

Epoch 36/50
1688/1688 ————— 9s 4ms/step - accuracy: 0.9975 - loss: 0.0088 - val_accuracy: 0.9823 - val_loss: 0.1367

Epoch 37/50
1688/1688 ————— 12s 5ms/step - accuracy: 0.9978 - loss: 0.0074 - val_accuracy: 0.9843 - val_loss: 0.1411

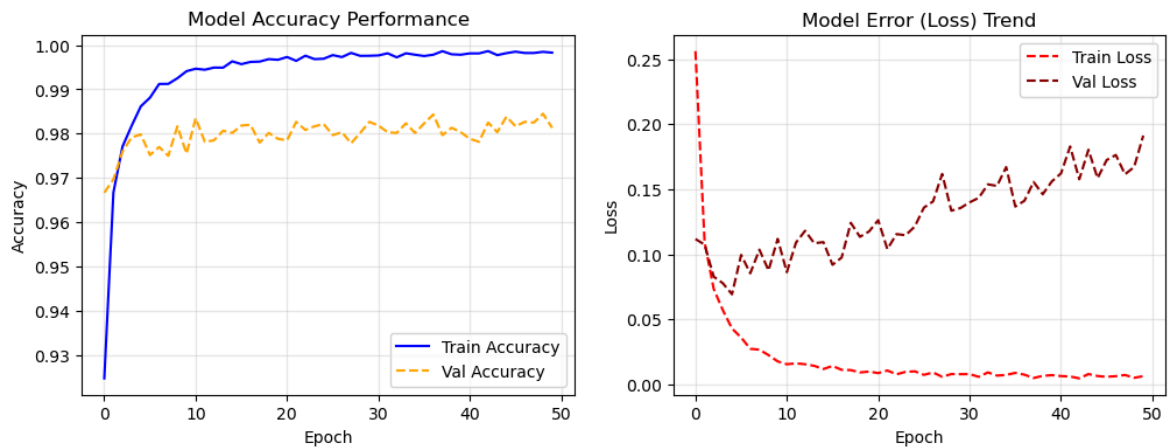
Epoch 38/50
1688/1688 ————— 8s 4ms/step - accuracy: 0.9986 - loss: 0.0048 - val_accuracy: 0.9797 - val_loss: 0.1556

Epoch 39/50
1688/1688 ————— 10s 4ms/step - accuracy: 0.9979 - loss: 0.0065 - val_accuracy: 0.9813 - val_loss: 0.1461

Epoch 40/50
1688/1688 ————— 6s 3ms/step - accuracy: 0.9978 - loss: 0.0070 - val_accuracy: 0.9803 - val_loss: 0.1560

Epoch 41/50
1688/1688 ————— **11s** 4ms/step - accuracy: 0.9981 - loss: 0.0063 - val_accuracy: 0.9788 - val_loss: 0.1622
 Epoch 42/50
1688/1688 ————— **7s** 4ms/step - accuracy: 0.9981 - loss: 0.0060 - val_accuracy: 0.9782 - val_loss: 0.1828
 Epoch 43/50
1688/1688 ————— **9s** 4ms/step - accuracy: 0.9986 - loss: 0.0046 - val_accuracy: 0.9825 - val_loss: 0.1576
 Epoch 44/50
1688/1688 ————— **12s** 5ms/step - accuracy: 0.9977 - loss: 0.0078 - val_accuracy: 0.9803 - val_loss: 0.1803
 Epoch 45/50
1688/1688 ————— **6s** 4ms/step - accuracy: 0.9982 - loss: 0.0064 - val_accuracy: 0.9838 - val_loss: 0.1585
 Epoch 46/50
1688/1688 ————— **10s** 4ms/step - accuracy: 0.9985 - loss: 0.0057 - val_accuracy: 0.9817 - val_loss: 0.1724
 Epoch 47/50
1688/1688 ————— **6s** 3ms/step - accuracy: 0.9982 - loss: 0.0063 - val_accuracy: 0.9827 - val_loss: 0.1763
 Epoch 48/50
1688/1688 ————— **12s** 4ms/step - accuracy: 0.9982 - loss: 0.0072 - val_accuracy: 0.9825 - val_loss: 0.1615
 Epoch 49/50
1688/1688 ————— **7s** 4ms/step - accuracy: 0.9985 - loss: 0.0050 - val_accuracy: 0.9845 - val_loss: 0.1670
 Epoch 50/50
1688/1688 ————— **8s** 5ms/step - accuracy: 0.9983 - loss: 0.0062 - val_accuracy: 0.9813 - val_loss: 0.1912

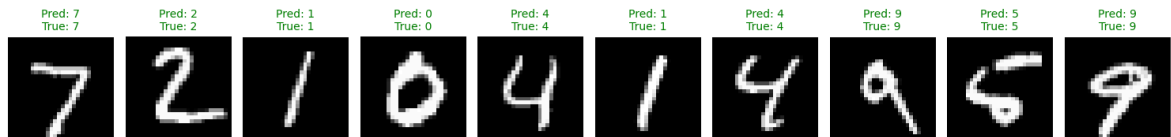
```
In [13]: plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', color='blue')
plt.plot(history.history['val_accuracy'], label='Val Accuracy', color='orange', linestyle='--')
plt.title('Model Accuracy Performance')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', color='red', linestyle='--')
plt.plot(history.history['val_loss'], label='Val Loss', color='darkred', linestyle='--')
plt.title('Model Error (Loss) Trend')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



```
In [16]: print("\nGenerating predictions on unseen test data...")
predictions = model.predict(X_test)
plt.figure(figsize=(15, 3))
for i in range(10):
    plt.subplot(1, 10, i+1)
    plt.imshow(X_test_images[i], cmap='gray')
    pred_label = np.argmax(predictions[i])
    true_label = y_test[i]
    color = 'green' if pred_label == true_label else 'red'
    plt.title(f"Pred: {pred_label}\nTrue: {true_label}", color=color, fontsize=10)
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Generating predictions on unseen test data...

313/313 ————— 1s 3ms/step



```
In [17]: loss, accuracy = model.evaluate(X_test, y_test_cat, verbose=0)
print(f"\n[FINAL REPORT] Test Loss: {loss:.4f}")
print(f"[FINAL REPORT] Test Accuracy: {accuracy*100:.2f}%")
```

[FINAL REPORT] Test Loss: 0.2233

[FINAL REPORT] Test Accuracy: 97.62%